

Application type: React personal portfolio, blog, gallery, resume, and AI assistant

Frontend: React 19, Vite, React Router, Tailwind CSS, Lucide React

Backend services: Netlify Functions, OpenAI API, Firebase Authentication, Firestore, GitHub API, Google Analytics 4

Deployment target: Netlify

Document date: September 13, 2026

1. Executive summary

Deepak Studio is a single-domain personal website that combines:

- * Public profile and biography pages.
- * Writing and project pages.
- * Gallery content.
- * Resume viewing and local upload support.
- * A Google-authenticated administrator workspace.
- * An OpenAI-powered portfolio assistant.
- * GitHub enrichment and optional LinkedIn profile context.
- * Google Analytics 4 realtime visitor reporting.

The application uses a Vite-built React frontend and Netlify serverless functions for operations that must not run in the browser, including OpenAI requests, resume text extraction, GitHub enrichment, and GA4 reporting.

2. Repository structure

```
text
my-blogs/
  netlify/
    functions/
      analytics-realtime.mjs
      portfolio-chat.mjs
  public/
  src/
    ai/
      PortfolioAssistant.jsx
  app/
    App.jsx
  features/
    admin/
```

```
public/  
lib/  
  analytics.js  
  firebase.js  
  storage.js  
styles/  
  App.css  
firestore.rules  
netlify.toml  
package.json  
.env.example
```

Main runtime flow

1. The browser loads the React application.
2. Public content is read from localStorage and environment-configured social links.
3. Admin login uses Firebase Google Authentication.
4. The admin workspace is additionally checked against VITE_ADMIN_EMAIL.
5. Chat requests go to /.netlify/functions/portfolio-chat.
6. The function validates the request, checks the topic, enriches public GitHub data, extracts text from a supplied PDF data URL, and calls OpenAI.
7. Realtime analytics requests go to /.netlify/functions/analytics-realtime.
8. The function validates the Firebase ID token before calling GA4.

3. Local development

Prerequisites

- * Node.js 20 or newer.
- * npm.
- * A Firebase project.
- * A Netlify account for deployed functions.
- * An OpenAI API key for live chatbot responses.

Install and run

```
powershell  
cd "C:\Users\Deepak Kumar Sharma\Desktop\AWS\my-blogs"  
npm.cmd install  
Copy-Item .env.example .env  
npm.cmd run dev:netlify
```

Open:

text
http://localhost:8888

Use the Netlify development command rather than npm run dev when testing functions. Plain Vite does not serve Netlify Functions and will produce chatbot 404 errors.

Validation commands

powershell
npm.cmd run lint
npm.cmd run build

4. Environment configuration

Environment variables are divided into browser-safe VITE_ values and server-only values.

Browser-safe frontend variables

Variable	Required	Purpose
VITE_FIREBASE_API_KEY	Yes	Firebase web configuration
VITE_FIREBASE_AUTH_DOMAIN	Yes	Firebase Authentication domain
VITE_FIREBASE_PROJECT_ID	Yes	Firebase project identifier
VITE_FIREBASE_STORAGE_BUCKET	Yes	Firebase Storage configuration
VITE_FIREBASE_MESSAGING_SENDER_ID	Yes	Firebase messaging configuration
VITE_FIREBASE_APP_ID	Yes	Firebase web app identifier
VITE_FIREBASE_MEASUREMENT_ID	Recommended	GA4 measurement identifier
VITE_ADMIN_EMAIL	Yes	Authorized Google account for the admin UI
VITE_CHAT_ENDPOINT	Optional	Defaults to /.netlify/functions/portfolio-chat
VITE_ANALYTICS_ENDPOINT	Optional	Defaults to /.netlify/functions/analytics-realtime
VITE_GITHUB_URL	Recommended	Public GitHub profile URL
VITE_LINKEDIN_URL	Recommended	Public LinkedIn profile URL

Server-only Netlify variables

Variable	Required	Purpose
OPENAI_API_KEY	Yes for AI	Secret used by the chatbot and health check
OPENAI_MODEL	Optional	Defaults to gpt-4o-mini
LINKEDIN_PROFILE_SUMMARY	Recommended	Curated LinkedIn facts; LinkedIn is not scraped automatically
FIREBASE_SERVICE_ACCOUNT_JSON	Required for analytics	Firebase Admin service-account JSON
GA4_PROPERTY_ID	Required for analytics	Numeric GA4 property ID

| ADMIN_EMAIL | Required for analytics | Server-side admin email allowlist |

Missing values to replace

The following placeholders must be replaced before production deployment:

- * Firebase web configuration values beginning with VITE_FIREBASE_.
- * VITE_ADMIN_EMAIL and ADMIN_EMAIL.
- * VITE_GITHUB_URL.
- * VITE_LINKEDIN_URL.
- * LINKEDIN_PROFILE_SUMMARY.
- * OPENAI_API_KEY.
- * FIREBASE_SERVICE_ACCOUNT_JSON.
- * GA4_PROPERTY_ID.

Do not commit .env. Do not place OPENAI_API_KEY or FIREBASE_SERVICE_ACCOUNT_JSON in variables beginning with VITE_; Vite exposes VITE_ values to the browser.

5. Firebase setup

1. Create or open the Firebase project.
2. Enable Google sign-in under Authentication Sign-in method.
3. Register a Web app and copy its configuration into the VITE_FIREBASE_* variables.
4. Create the Firestore database.
5. Publish [firestore.rules](../firestore.rules).
6. Replace the administrator email in the rules with the exact Google account used by the owner.
7. Add the production Netlify domain to Firebase Authentication authorized domains.

The current Firestore rules allow public reads for the legacy AI settings document and restrict writes to the configured admin email. The admin UI no longer exposes an AI availability toggle, so that document is not required for chatbot operation.

6. OpenAI chatbot configuration

The chatbot function is:

text
/.netlify/functions/portfolio-chat

Health check

text

GET /.netlify/functions/portfolio-chat

- * HTTP 200 with {"online":true} means the server-side key was accepted by OpenAI.
- * HTTP 503 means the key is missing, invalid, expired, or unavailable.

Chat request

text

POST /.netlify/functions/portfolio-chat

The function uses:

- * Website profile, posts, projects, and gallery metadata.
- * Public GitHub profile and recent repositories.
- * LINKEDIN_PROFILE_SUMMARY.
- * Extracted text from the locally uploaded PDF resume.

Resume input is restricted to base64 PDF data URLs. Remote URLs are rejected to prevent server-side request forgery.

7. Netlify deployment

First deployment through the Netlify dashboard

1. Push the project to a Git provider.
2. Sign in to Netlify.
3. Select Add new site Import an existing project.
4. Choose the repository.
5. Set the build command to npm run build.
6. Set the publish directory to dist.
7. Confirm the functions directory is netlify/functions.
8. Add all frontend variables under Site configuration Environment variables.
9. Add all server-only variables under the same Netlify environment settings.
10. Deploy the site.
11. Add the deployed domain to Firebase Authentication authorized domains.
12. Publish Firestore rules.
13. Test the website, /admin, the chatbot, and the analytics endpoint.

netlify.toml already declares:

toml

[build]

command = "npm run build"

publish = "dist"

```
functions = "netlify/functions"
```

```
[functions]  
node_bundler = "esbuild"
```

CLI deployment option

```
powershell  
npm.cmd install  
npx netlify login  
npx netlify link  
npx netlify env:import .env  
npm.cmd run build  
npx netlify deploy --prod --dir=dist
```

Review imported variables carefully. Never import a local .env containing secrets into an untrusted site.

Post-deployment checklist

- * Open the production homepage.
- * Confirm assets load over HTTPS.
- * Open /admin/login.
- * Sign in with the authorized Google account.
- * Confirm an unauthorized Google account is rejected.
- * Open the chatbot and verify its health status.
- * Ask a portfolio question and confirm the Netlify function returns an answer.
- * Open the GA4 realtime panel while authenticated.
- * Confirm invalid or missing OpenAI keys produce an offline state.
- * Confirm the browser never contains OPENAI_API_KEY or service-account JSON.

8. Security controls

- * Admin routing validates the current Firebase-authenticated user instead of trusting a session flag.
- * Firestore writes are restricted by email in rules.
- * OpenAI credentials remain server-side.
- * Resume remote URL fetching is disabled to prevent SSRF.
- * Chat topic filtering blocks unrelated questions.
- * Resume payloads have size limits.
- * Markdown links in chatbot responses are rendered with rel="noreferrer".
- * GitHub enrichment uses public API data only.
- * LinkedIn is represented by a curated summary rather than unauthenticated scraping.

Operational security recommendations

- * Rotate API keys immediately if they have ever been committed or pasted into public logs.
- * Use separate development and production OpenAI keys.
- * Set Netlify environment variable scopes explicitly.
- * Keep Firebase authorized domains limited to the production and approved preview domains.
- * Review Firebase rules after every data-model change.
- * Enable billing alerts and usage limits for OpenAI and Google Cloud.
- * Run dependency audits before production releases.

9. Troubleshooting

Chatbot returns 404

Use `npm.cmd run dev:netlify`, not plain Vite. Confirm `netlify.toml` exists and the function is under `netlify/functions`.

Chatbot is offline

Check `OPENAI_API_KEY` in Netlify environment variables. Redeploy after changing it. Test the GET health endpoint.

Admin redirects to login

Confirm Google sign-in is enabled, the current email matches `VITE_ADMIN_EMAIL`, and the deployed domain is an authorized Firebase domain.

Analytics shows no realtime users

Confirm `FIREBASE_SERVICE_ACCOUNT_JSON`, `GA4_PROPERTY_ID`, and `ADMIN_EMAIL` are configured as Netlify server variables. Enable the Google Analytics Data API for the service account project and grant the service account Viewer access to the GA4 property.

Local Netlify server returns 500

Stop stale Vite/Netlify processes, then run one clean `npm.cmd run dev:netlify` process on port 8888. Ensure the `.env` file contains valid Firebase values.

10. Release checklist

- * ☐ `npm.cmd run lint` passes.
- * ☐ `npm.cmd run build` passes.
- * ☐ No secrets are tracked by Git.
- * ☐ Production environment variables are configured.

- * ☐ Firebase authorized domains are configured.
- * ☐ Firestore rules are published.
- * ☐ OpenAI health endpoint returns the expected status.
- * ☐ Admin authorization is tested with allowed and denied accounts.
- * ☐ Chatbot scope guard is tested.
- * ☐ Analytics authentication is tested.
- * ☐ Netlify deploy preview and production site are checked.